SmartCampus DBMA

Technical Specification & AI Implementation Guide

---

1. Core Philosophy (What You're Actually Building)

You are NOT building:

- "modules with pages"

You ARE building:

- **independent domains with business logic ownership**

Each domain must be:

- Self-contained
- Replaceable
- Testable
- API-exposed

---

2. Final Architecture Pattern (IMPORTANT)

We will use:

 DDD + Clean Architecture + Modular Monolith (for now)

2.1 Domain Structure Standard (MANDATORY)

Every domain MUST follow this:

```
/domain-name
  /domain          ← Pure business logic (NO framework)
  /application     ← Use cases (services)
  /infrastructure  ← DB, APIs, external services
  /presentation    ← API (Django views / serializers)
```

2.2 Your Real System Structure

```
src/
│
├── common/                  # shared kernel
│
├── domains/
│   ├── users/
│   ├── academics/
│   ├── finance/
│   ├── operations/
│   ├── inventory/
│   ├── communication/
│   ├── analytics/
│
├── config/                  # settings
├── main.py / manage.py
```

---

3. Deep Domain Breakdown (Your System)

Users Domain (CRITICAL FOUNDATION)

Domain Layer

```
entities/
  Student.py
  Parent.py
  Staff.py

value_objects/
  Email.py
  Phone.py

aggregates/
  StudentAggregate.py
```

Application Layer

```
use_cases/
  CreateStudent.py
  LinkParentToStudent.py
  RegisterParent.py
```

Infrastructure

```
repositories/
  DjangoStudentRepository.py
  DjangoParentRepository.py
```

Academics Domain

Owns:

- Enrollment (VERY IMPORTANT)
- Classes
- Grades

- This domain is the **core of your system**

Finance Domain

RULE:

Finance must NEVER control Students or Academics

It only:

- references `student_id`
- references `term_id`

- Inventory Domain

Completely isolated:

```
Item → Stock → Movement
Asset → Assignment
```

- Operations Domain

Each sub-module can later become its own microservice:

- Library
- Transport
- Hostel
- Cafeteria

---

4. Domain Communication Rules (VERY IMPORTANT)

4.1 Allowed Communication

From	To	How
Finance	Users	Read only
Finance	Academics	Read only
Academics	Users	Full
Operations	Users	Read

4.2 Forbidden

 Direct DB access across domains
 Importing models from another domain

4.3 Correct Way

Use:

- Interfaces
- DTOs

Example:

```
Finance → StudentDTO
```

---

5. AI + Replit Workflow

You already said it — now let’s formalize it.

5.1 PLAN MODE (Blueprint Stage)

Prompt:

```
Create a Domain-Based Architecture for a School Management System.

Domains:
- Users
- Academics
- Finance
- Operations
- Inventory

Each domain must contain:
- domain/
- application/
- infrastructure/
- presentation/

DO NOT write full implementation.
ONLY generate:
- folder structure
- interfaces
- base classes
```

5.2 SKELETON PHASE (CHEAP + SAFE)

Prompt:

```
Generate the folder structure and empty classes for the Users domain.

Include:
- Entities
- Repository interfaces
- Use case interfaces

DO NOT implement logic.
```

5.3 DOMAIN LOGIC PHASE (EXPENSIVE – USE POWER MODEL)

Prompt:

```
Implement business logic for:

Use case: Enroll Student

Rules:
- Student must exist
- Class must exist
- Cannot enroll twice

Follow clean architecture strictly.
```

---

6. Cost Optimization Strategy (REAL WORLD)

Use AI Modes Like This:

Task	Mode
Folder structure	Economy
DTOs	Economy
CRUD APIs	Economy
Business rules	□ Power
Complex flows	□ Power

Golden Rule:

NEVER ask AI to "build full system"

Always:

- Build domain by domain

---

7. Database Strategy (Replit)

7.1 Use PostgreSQL (NOT Replit KV for this system)

- Multi-tenant support
- Relationships needed

7.2 Repository Pattern (MANDATORY)

Example:

```
class StudentRepository:
    def get_by_id(id): pass
    def save(student): pass
```

Then:

```
class DjangoStudentRepository(StudentRepository):
```

 This allows:

- DB swap later
- Clean domain logic

---

8. Security & Secrets

Use Replit Secrets for:

- DB credentials
- JWT secret
- API keys

 NEVER inside:

```
/domain
```

---

9. Deployment Strategy

Use Replit Deployments:

- Version 1 → stable
- Version 2 → testing new domain

---

10. Common Mistakes (YOU MUST AVOID)

Mistake 1: Fat Models (Django default)

Fix: move logic to domain layer

Mistake 2: Cross-domain imports

Fix: use interfaces

Mistake 3: AI generating everything at once

Fix: staged prompting

Mistake 4: Mixing concerns

Example:

- Finance calculating grades
- Academics handling payments

---

11. Implementation Roadmap (FOR YOU)

Phase 1: Skeleton (ALL DOMAINS)

- Create all domain folders
- Define interfaces

Phase 2: Core Domains First

1. Users
2. Academics
3. Finance

Phase 3: Supporting Domains

4. Inventory
5. Operations
6. Communication

Phase 4: Analytics (LAST)

---

12. Testing Strategy (DDD Style)

Test per layer:

Domain Tests

- Business rules

Application Tests

- Use cases

Infrastructure Tests

- DB operations